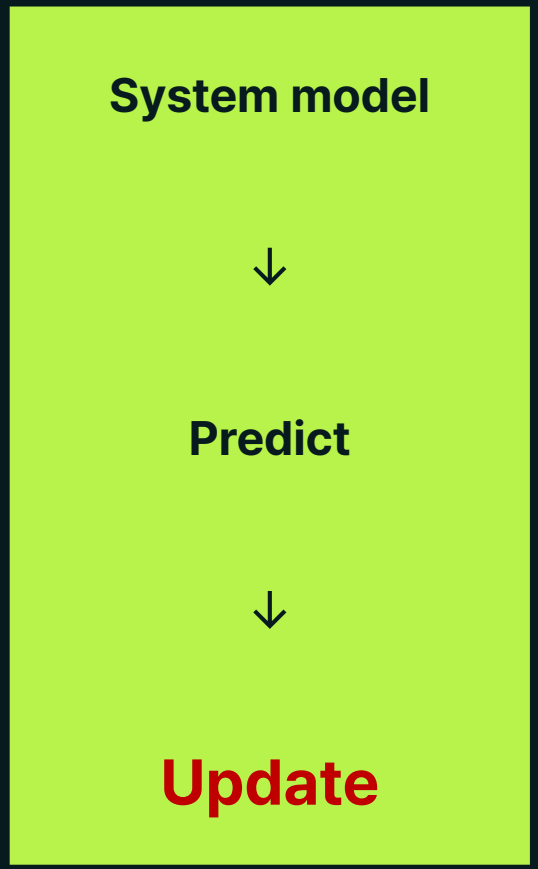
The Kalman Filter: The Basic Model

Two equations, two steps, repeated



Kalman filter: a hidden state from noisy data

The big picture — write the problem as two equations, solve it by repeating two steps

Example: tracking a car's position



True position = state x_k
cannot be observed directly

Task: estimate the true position from noisy readings

Kalman filter = two equations + two steps

System model (two equations)
How the state moves, and how it is measured

Step 1: Predict
Use the rule to guess the next state

Step 2: Update
Correct the guess with the measurement

Repeat with every new measurement

First the two equations (the system model), then the two steps

State transition: how the state moves over time

System model ① — the state transition equation

$$x_k = A\,x_{k-1} + B\,u_{k-1} + w_{k-1}$$

$A\,x_{k-1}$

Move by the rule

Apply rule A to yesterday's state
e.g. position + velocity × time

$B\,u_{k-1}$

External input

Force applied from outside
e.g. gas pedal — 0 in finance

w_{k-1}

Process noise

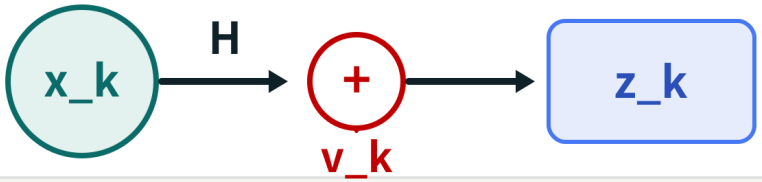
Wobble the rule cannot capture
Mean 0, size (variance) Q

Next state = state moved by the rule + external input + noise

Measurement: how the hidden state becomes data

System model ② — the measurement equation

$$z_k = H\, x_k + v_k$$



$H\, x_k$

To measurement units

Maps state to sensor units
e.g. position → GPS readout

v_k

Measurement noise

Error from the sensor
Mean 0, size (variance) R

z_k

Measurement

The number we actually see
Input to the Kalman filter

Measurement = state converted to measurement units + noise

The model uses only ten symbols

Colours are fixed across the deck — teal: state, blue: measurement, amber: process noise, red: measurement noise

x_k

State

Unknown value we want to know

z_k

Measurement

Number seen from sensors or data

A

Transition matrix

Rule moving the state one step ahead

H

Measurement matrix

Rule mapping the state to readings

u_{k-1}

Control input

Outside force (none in finance)

Q

Process noise

How much the state wobbles

R

Measurement noise

How wrong the readings are

P

Uncertainty

How shaky our guess is

K

Kalman gain

How much of the reading to accept

$\hat{x}_k^-$, $\hat{x}_k$

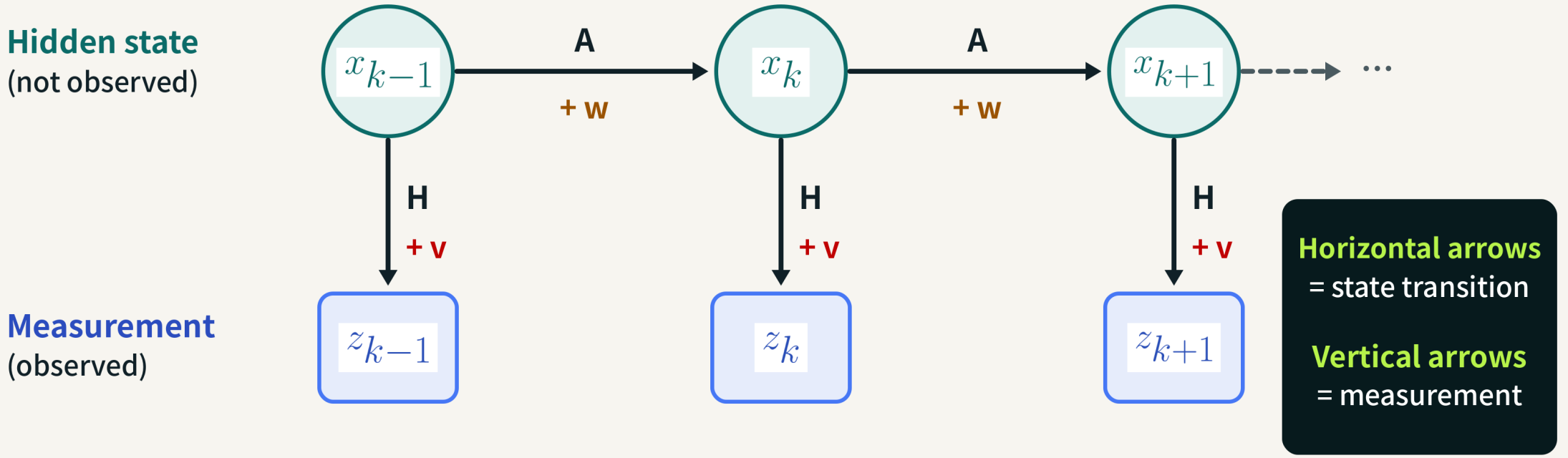
Prior · posterior

Guess before and after measuring

Every equation that follows is written with these ten symbols only

Two equations: a chain of states, a row of data

Top row: unobservable states. Bottom row: observable measurements



The Kalman filter sees only the bottom row and infers the top row from it

Step 1, predict: move by the rule, add uncertainty

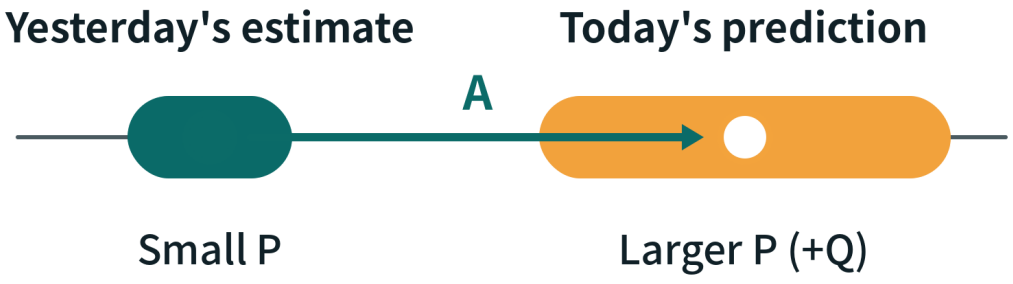
Predict — the calculation done before the measurement arrives

$$\hat{x}_k^- = A\,\hat{x}_{k-1} + B\,u_{k-1}$$

$$P_k^- = A\,P_{k-1}A^T + Q$$

① Move yesterday's estimate by rule A
② Carry uncertainty forward, add Q

Measurement z is not used yet



Both equations come straight from the state transition equation

Step 2, update: correct the prediction with data

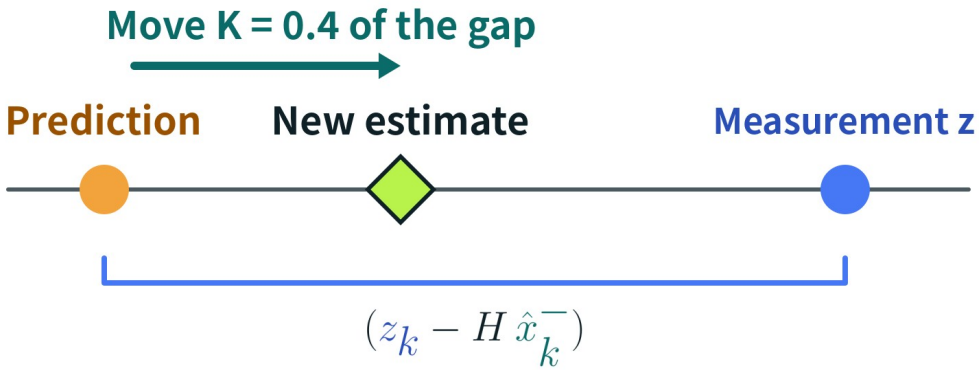
Update — compute the gain K, then move by K times the innovation

$$K_k = P_k^- H^T (H P_k^- H^T + R)^{-1}$$

$$\hat{x}_k = \hat{x}_k^- + K_k (z_k - H\,\hat{x}_k^-)$$

$$P_k = (I - K_k H)\, P_k^-$$

③ Compute gain K ④ Correct by K × innovation
⑤ Shrink uncertainty P



Innovation = how far the reading misses the prediction

New estimate = prediction moved toward the data by K × innovation

The Kalman gain weighs prediction against data

The gain in a scalar model (H = 1) — the amber share of each bar is K

$$K = \frac{P^-}{P^- + R} \quad (H = 1)$$

In the scalar model (H = 1),
K = the prediction's share
of the total uncertainty

K → 0: trust the prediction
K → 1: trust the measurement

When the prediction is uncertain



K = 4 ÷ (4 + 1) = 0.8 → accept 80% of the innovation

When the measurement is noisy

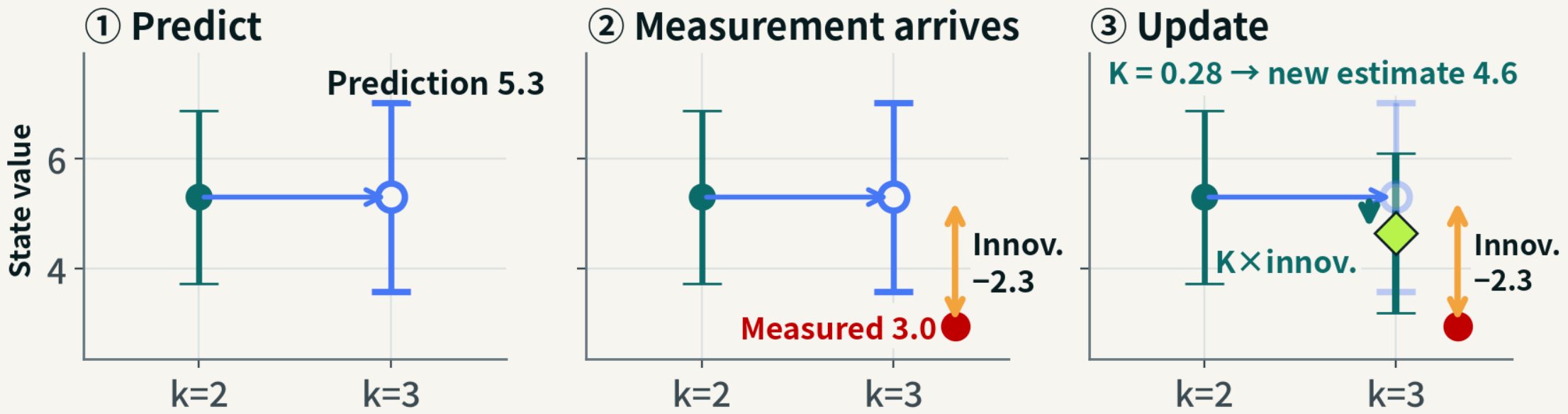


K = 1 ÷ (1 + 4) = 0.2 → accept 20% of the innovation

The gain is not chosen; it is computed from P and R

Predict and update at a single time step

Zoom on k = 3 — prediction 5.3, measurement 3.0, gain 0.28

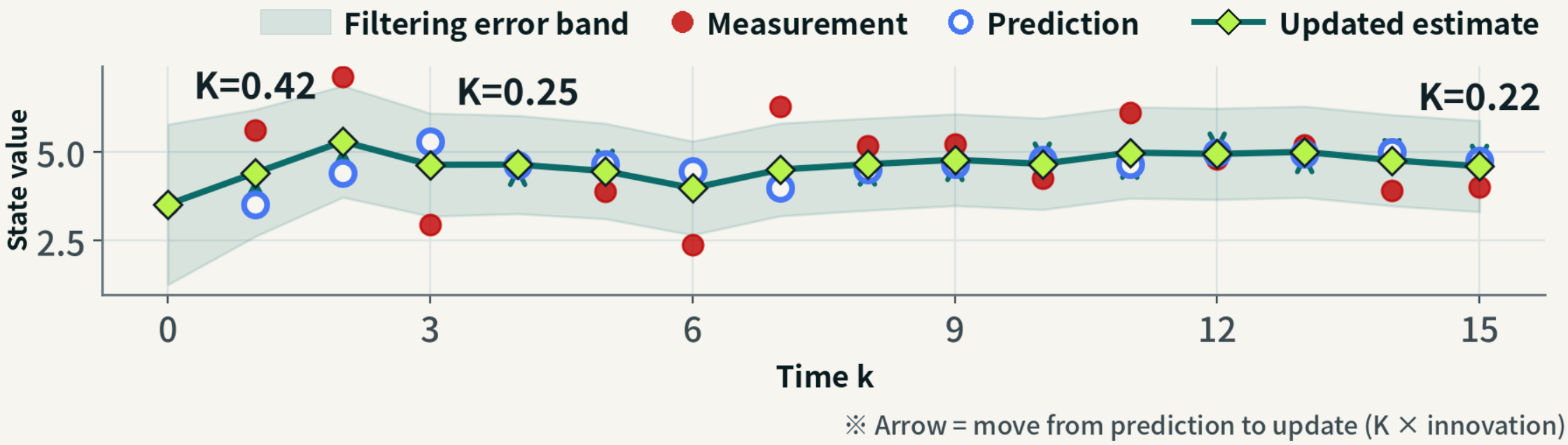


※ Bars = 95% error band (±1.96√P); same simulated data as the Python example

New estimate 4.6 = 5.3 + 0.28 × innovation (−2.3); the band narrows

Repeating the two steps draws the estimate forward

First 16 steps — open circles are predictions, diamonds are updated estimates

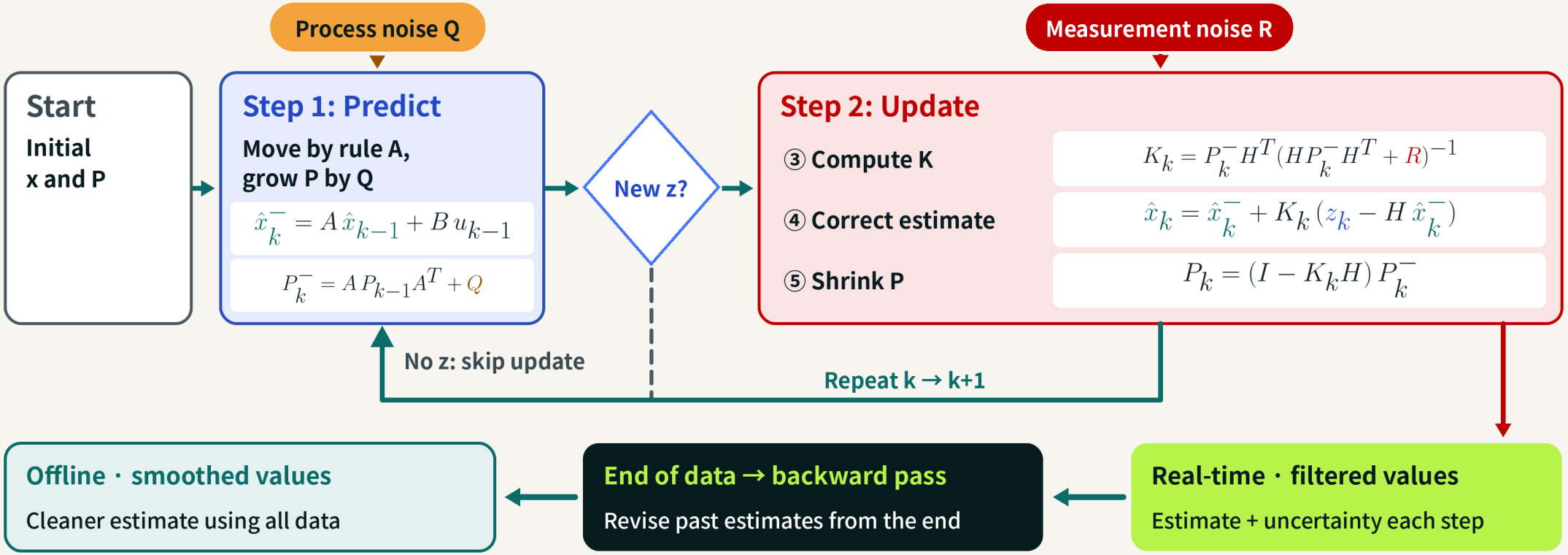


※ Arrow = move from prediction to update (K × innovation)

The gain starts large, so early moves are big; it soon settles to a steady value

The whole procedure is one loop from initial values

Workflow — each new measurement triggers the two predict equations, then the three update equations



The Python code is the five equations, line by line

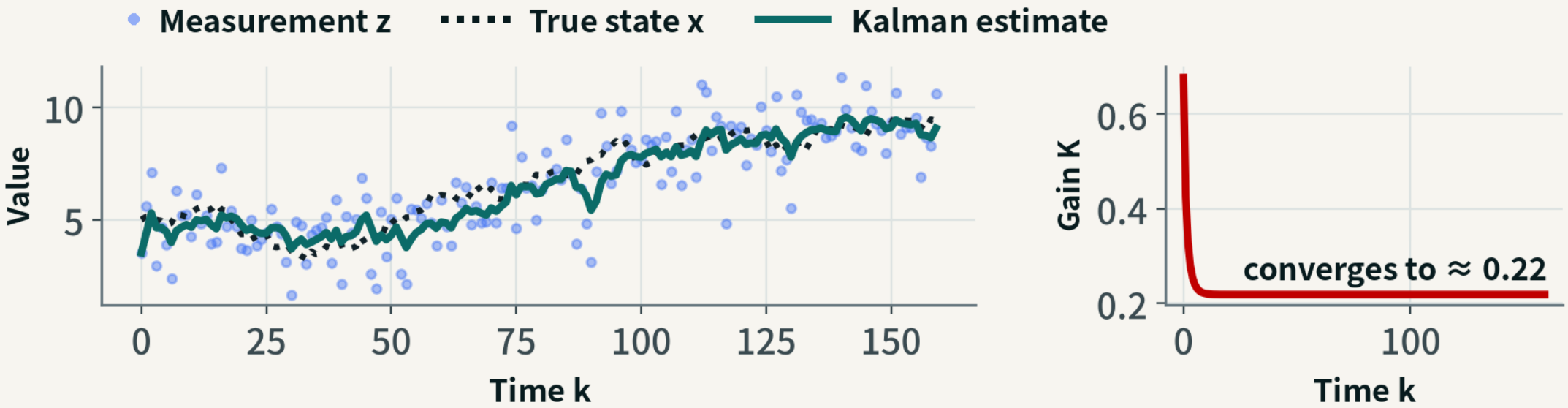
numpy only — the right column shows the equation each line computes (scalar model, no control input)

```
A, H = 1.0, 1.0
Q, R = 0.12, 1.96
x, P = z[0], 4.0
for zk in z:
    x_pred = A * x
    P_pred = A * P * A + Q
    K = P_pred*H / (H*P_pred*H + R)
    x = x_pred + K * (zk - H*x_pred)
    P = (1 - K*H) * P_pred
```

Rule	$A = 1, \quad H = 1$
Noise	$Q, \quad R$
Initial values	$\hat{x}_0, \quad P_0$
①	$\hat{x}_k^- = A\,\hat{x}_{k-1} + B\,u_{k-1}$
②	$P_k^- = A\,P_{k-1}A^T + Q$
③	$K_k = P_k^- H^T (H P_k^- H^T + R)^{-1}$
④	$\hat{x}_k = \hat{x}_k^- + K_k\,(z_k - H\,\hat{x}_k^-)$
⑤	$P_k = (I - K_k H)\,P_k^-$

The filter recovers the true state from noisy data

The code above run on 160 simulated measurements — right: the gain K at each step

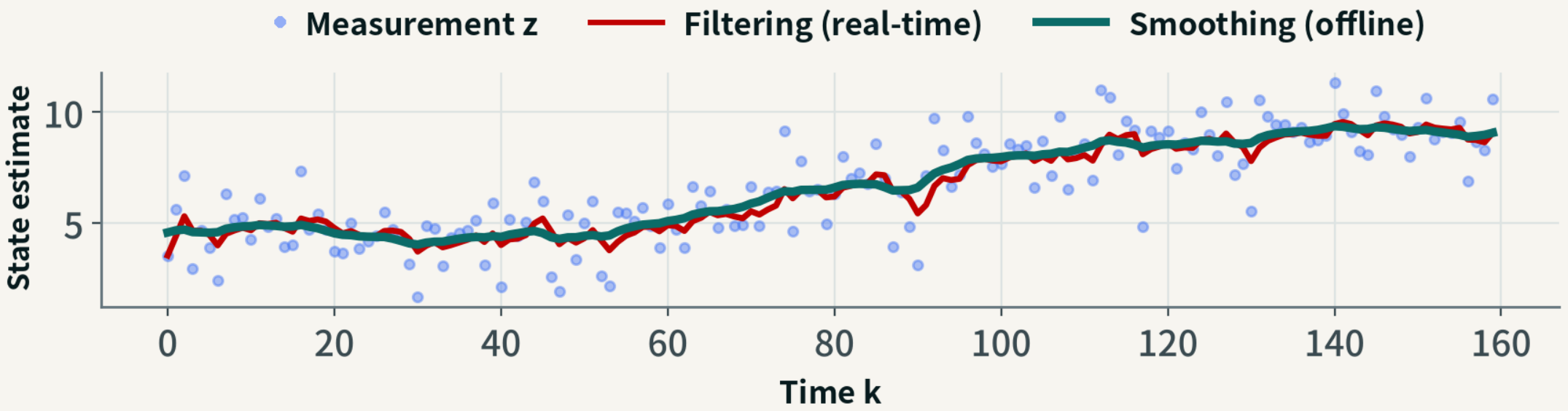


※ Simulated data · Q = 0.12, R = 1.96 · RMSE: measurement 1.33 → Kalman 0.66

Estimation error falls to less than half of the measurement error

A backward pass after the data end gives smoothing

Filtering (real time, one forward pass) vs smoothing (after the fact, forward + backward)



※ Simulated data example

Trading uses filtered values — smoothed values come only afterwards

Same equations as the lecture PDF, different letters

Notation map — watch the red rows, where the letters differ

This deck (basic-model notation)		Lecture PDF notation
State	x_k	x_t
State transition matrix	A	T
Measurement matrix	H	Z
Measurement	z_k	y_t
Process noise	Q	Q
Measurement noise	R	H
Innovation (meas. − pred.)	z_k − H · prediction	v_t
Time	k	t

Caution

Lecture PDF: H = measurement noise

This deck: H = measurement matrix

Same letter, different meaning

The form and order of the equations are exactly the same

The same estimates in conditional notation

x̂(t|s) = estimate of the state at time t using data up to time s

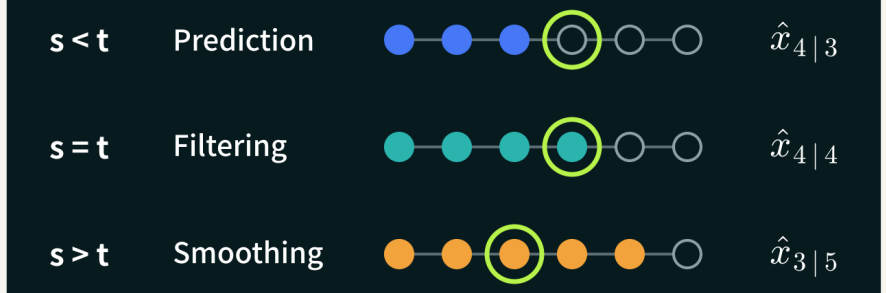
This deck	Conditional	Read as	Name
$\hat{x}_k^-$	$\hat{x}_{k\,\vert\,k-1}$	state at k, data up to k−1	Prediction (prior)
$\hat{x}_k$	$\hat{x}_{k\,\vert\,k}$	state at k, data up to k	Filtering (posterior)
—	$\hat{x}_{k-1\,\vert\,k}$, $\hat{x}_{k\,\vert\,T}$	past state, later data	Smoothing
P_k^-, P_k	$P_{k\,\vert\,k-1}$, $P_{k\,\vert\,k}$	variance of each estimate	Uncertainty

How to read it

$\hat{x}_{t\,\vert\,s}$

t : the time whose state we estimate

s : the last time whose data we use



filled = data used ring = time estimated

Prediction is k|k–1, filtering is k|k, smoothing is k–1|k or k|T

Two equations: state transition and measurement.

Two steps: predict and update.

The gain K sets the weight between them.

Next: what do we plug into
these slots to get applications?